UVUnify Atlas Tool

A Unity Editor Tool for Texture Atlasing, Mesh Combining, Packed Masks, LODs, and Prefab Optimization

---

Created by:
Walter Sattazahn
UVUnify Studio Tools

Compatible With:
Unity 2020.3+ · Standard / URP / HDRP Pipelines
Editor-Only Tool · No Runtime Components

Includes Features Like:

- Texture atlasing with padding and fallback support
- Packed Mask generation (RGBA: Metallic, AO, Height, Emission)
- Transparent and opaque group handling
- LOD simplification with custom quality controls
- UV2 lightmapping support for URP
- Clean prefab creation and export folders
- Summary reports and batch asset processing

Manual Version: v1.0
Last Updated: May 2025

For support and updates, visit:
waltsdesigns.com | *UVUnify Tool Manual*

UVUnify Atlas Manual

Table of Contents

Next Page:

On the next page starts a *Quickstart Overview* showing you the basics for creating a **combined mesh** that uses the same material and atlases, and then to generate an **LOD** from the combined mesh.

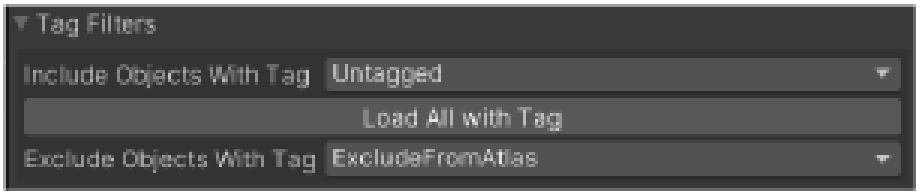
Quickstart Overview

> ***Follow these steps to generate optimized atlases and prefabs.***

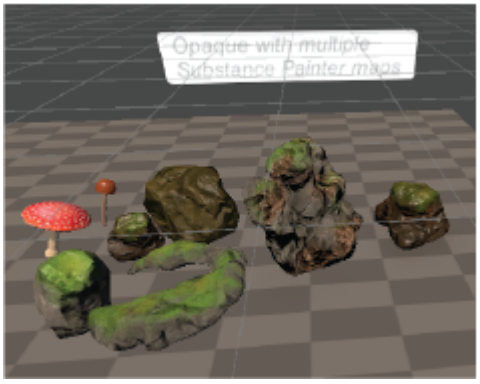
Step 1: Load Your Objects

- Select one or more GameObjects in your scene
- Click **"Load Selected GameObjects"**

Optional: Filters
You can avoid loading unwanted objects by
“Excluding Objects with Tag”.



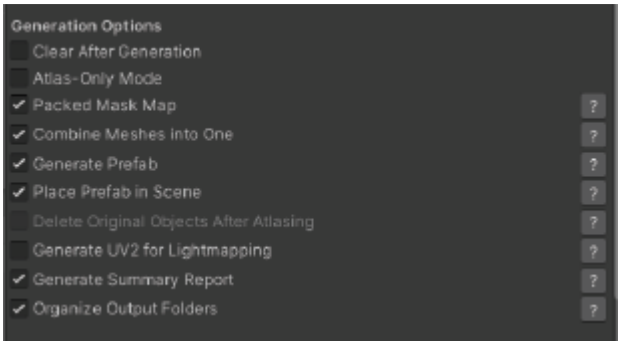
Selected objects



Load Selected GameObjects Button

Step 2: Enable Features

- **Enable: Combine Meshes into One**
  (Required for prefab creation and ideal before generating LODs)
- **Enable: Generate Prefab**
  (Saves the combined result as a reusable asset)
- **Enable: Place Prefab in Scene**
  (Instantiates the prefab into your current scene after generation)
- **Enable: Packed Mask Map**
  (Automatically combines Metallic, AO, Height, and Emission into one optimized texture — URP or HDRP only)
- **Enable: Organize Output Folders**
  (Helps keep your project clean by saving assets into subfolders)
- **Enable: Generate Summary Report**
  (Saves a readable .txt log of your generation settings and results)



Optional: Use Packed Mask Map *(optional)*

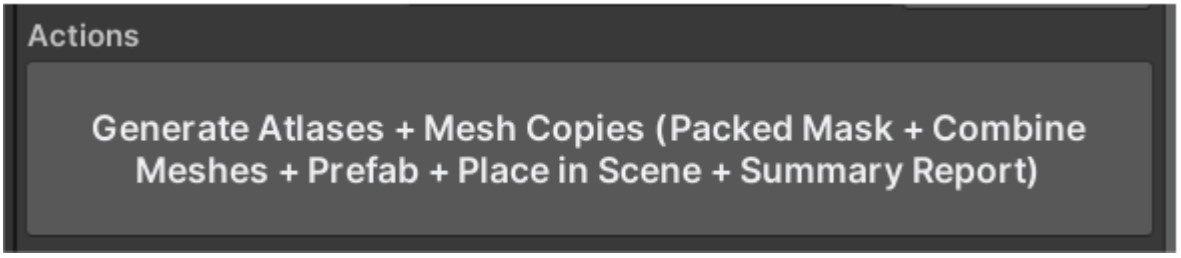
- For HDRP/URP, check **Packed Mask Map**
- It will auto-detect **Metallic, AO, Height**, and **Emission**

Step 3: Choose the Save Folder

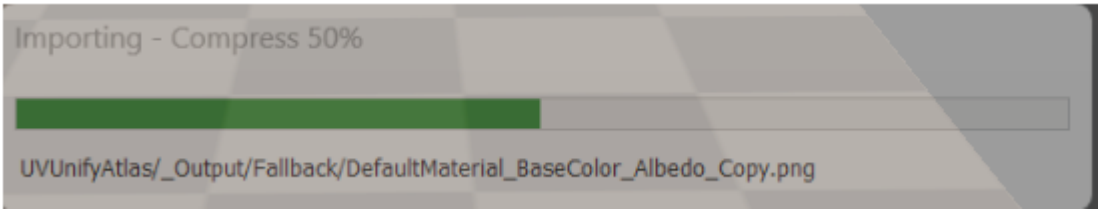
- Make sure everything you generate goes into the folder that you want.

Step 4: Generate

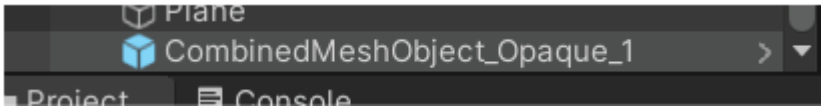
- The dynamic button will reflect the toggles you have ON.



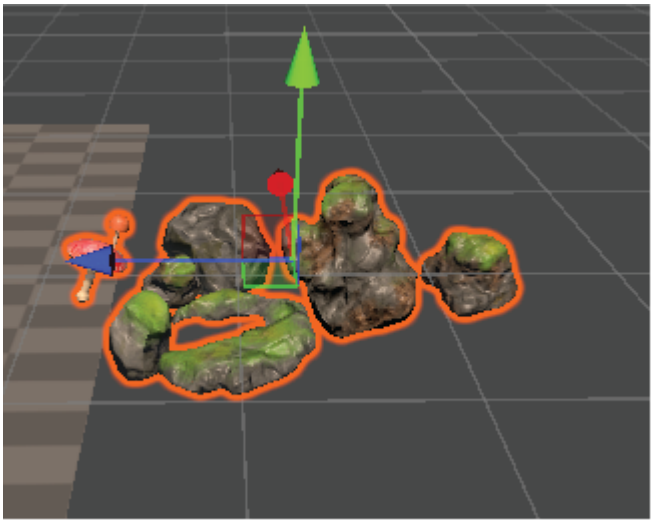
Unity will show a loading bar as the assets are being generated.



You will see a prefab is placed in the scene and the combined object will be in the place of the original objects.



Final Result: Combined Mesh + Output Folders



Here are all the objects combined as **one mesh** with their details preserved.
They now share the same combined material and atlases.

The assets generated for **Opaque** and **Transparent** objects are separated.

Save Folder Structure

In the Save Folder, we can see the **organized subfolders** that were generated:

- **Atlases_Opaque / Atlases_Transparent**:
  Contains all texture atlases generated from your selected objects.
- **TextureCopies**:
  Safe, readable duplicates of your source textures (e.g., Albedo, Normal).
- **Materials**:
  Contains the combined material created during generation.
- **Meshes**:
  Contains the combined mesh asset (if enabled) and simplified LOD meshes (if enabled).
- **Prefabs**:
  Stores prefabs created during generation.
- **Reports**:
  Contains a `.txt` summary report (if the option is enabled).

Quickstart: LOD

Follow these steps to generate optimized atlases and prefabs.

LOD

Now let's make a **LOD (Level of Detail)** with the combined mesh.

Step 1: Clear Selected Objects

- Clear the original objects that were loaded into UVUnify Atlas using the **Clear Selected Objects** button.
- To do this automatically after generating, toggle ON **Clear After Generation** under the Generation Options.

Step 2: Load the Combined Mesh

- Select the combined mesh you just created and press **Load Selected GameObjects**.

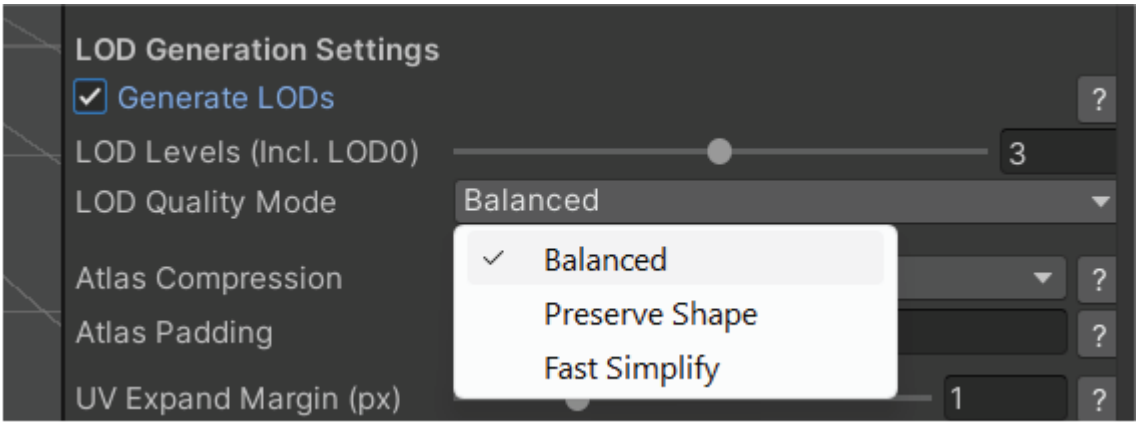
Optional: Refresh

- Press **Refresh Selected Objects** if the object type is not matching expectations (e.g. an opaque object is marked as transparent).
- You can also toggle on **Force Opaque** beside an object's name if you don't want it treated as transparent.

Step 3: Toggle ON Generate LODs

- Enables **mesh simplification** (Level of Detail).
- Choose between:
  - **Balanced**: Smart average

- **Preserve Shape**: Highest fidelity
- **Fast Simplify**: Prioritizes speed



Quickstart: LOD (Continued)

LOD Levels (Incl. LOD0)

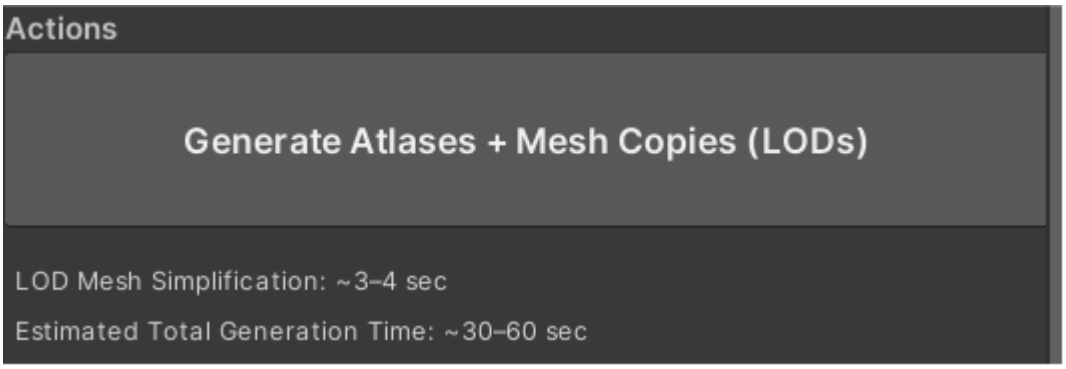
- Sets the total number of LOD levels to generate:
  - **LOD0** = original mesh
  - **LOD1, LOD2, etc.** = progressively simplified versions
  - **Recommended:** 3 levels for general use

LOD Quality Mode

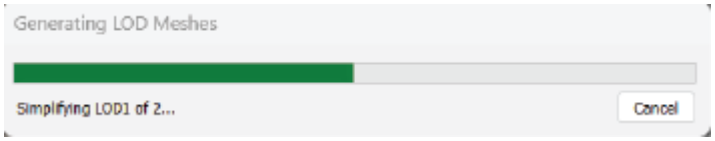
Mode	Description
Balanced	Default. Optimizes both shape and performance. Good for most use cases.
Preserve Shape	Maintains silhouette and curvature. Ideal for stylized or organic objects.
Fast Simplify	Quickest method. Prioritizes speed and triangle reduction. May look harsher.

Step 3: Generate Atlases + Mesh Copies (LODs)

- Adaptive estimated time is based on system specs and previous LOD run.



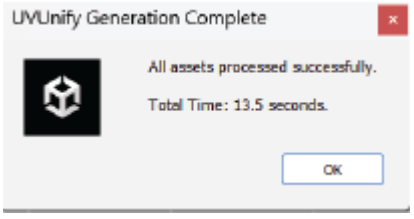
> *Wait for LOD to complete.*



Quickstart: LOD Completion

LOD Generation Complete

A popup notifies you when the LOD is complete!



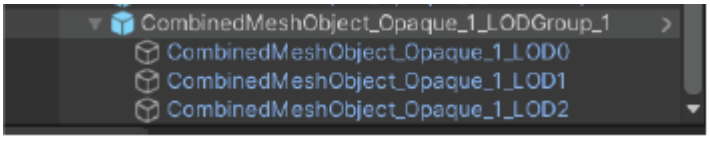
LOD Vertex Breakdown

The LOD Breakdown shows how much the original combined mesh has been reduced for LOD1 and LOD2.

If you generate a summary report, you can also see how many draw calls have been reduced.

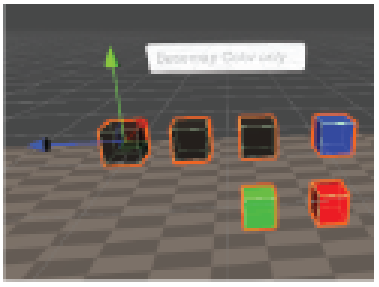
Scene View of LODs

A prefab of the generated LOD is placed in the scene in the same location as the original object. You can observe the LOD children and how they all use the same generated material with atlases.



Features: Support for Base Map Color Only Materials

🖌️ Objects with materials that have no textures but use a color for Base Map



Combining Objects with Base Map Color Only in Material

UVUnify will generate flat color textures and pack them into the albedo atlas for objects that have colors but no textures.
The textures will also not be duplicated—so if you have 3 objects with the same color, they will all use the same flat color texture in the atlas.

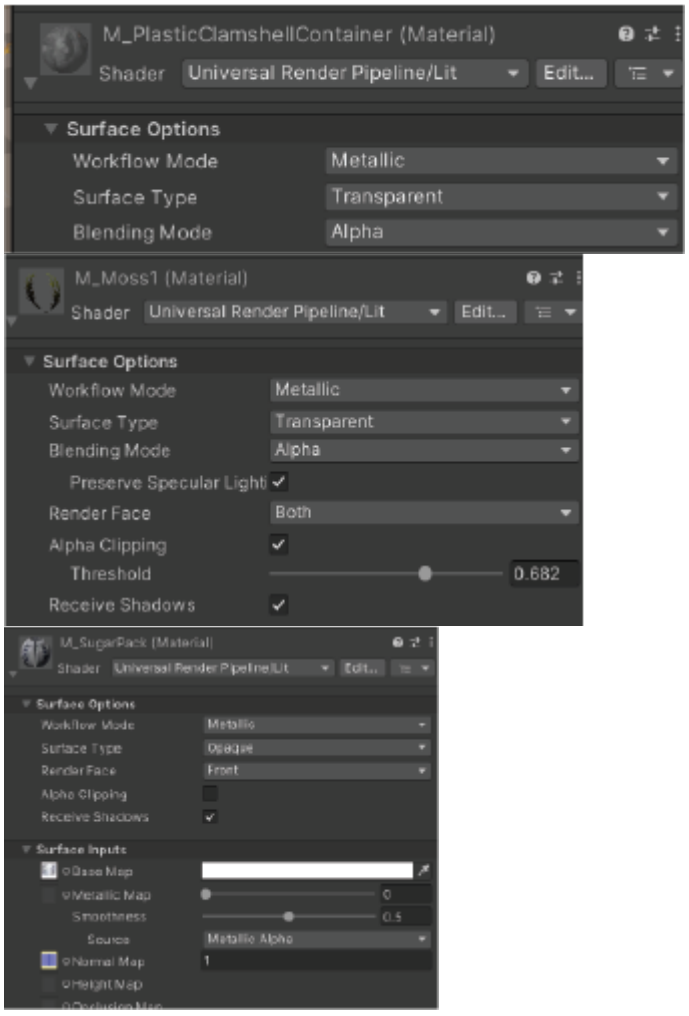
These types of objects are flagged with **(Missing Texture)** but their flat colors will still be generated and packed in atlases.

In this example where multiple objects with textures are generated, 3 plastic utensils have no textures but have a black Base Map color in their material.
One flat color texture for all 3 utensils is packed!

Note: There is a message that appears and a button where you can remove the objects with missing textures.
You can ignore it if you want to generate the Base Map color texture for that object.
: Transparent Object Detection

🔍 Transparent objects are detected in 3 ways:

- **Texture Transparency (Alpha Channel)**
  The object's base texture contains pixels with partial transparency (alpha < 255).
  Example: A leaf texture with soft edges.
- **Transparent Material Settings**
  The material is set to use transparency in the shader —
  like URP's `"Surface Type: Transparent"`, HDRP, or Standard shaders using transparency modes.
- **Alpha Clipping / Cutout Shader**
  The material uses alpha cutoff or a "cutout" shader, which removes parts of the mesh using a sharp alpha threshold.
  Example: A chain-link fence or plant with hard-edged transparency.



: Transparent Object Detection and Combining

🧩 Combining Transparent Objects

UVUnify supports combining all types of transparent objects, but for best results:

- **Cutout transparent objects**
  (using alpha clipping) should be combined separately from
- **Fully transparent objects**
  (using Surface Type: Transparent materials).

This separation helps avoid rendering issues like incorrect depth sorting or visual artifacts.
Transparent objects will always be separated from opaque objects.
If you do not want a specific object to be transparent, you can force it to be opaque with the toggle that appears beside its name.

When transparency is detected, UVUnify reveals two helpful toggles:

- **Auto Set Material To Transparent**
  This toggle automatically makes the combined material generated transparent.
- **Use Alpha Clipping Cutout**
  Alpha clipping can be set before generation or manually set in the material after generation
  and is recommended for objects like plants and foliage.
  : Packed Masks for URP and HDRP Pipelines

🟠 URP Packed Masks

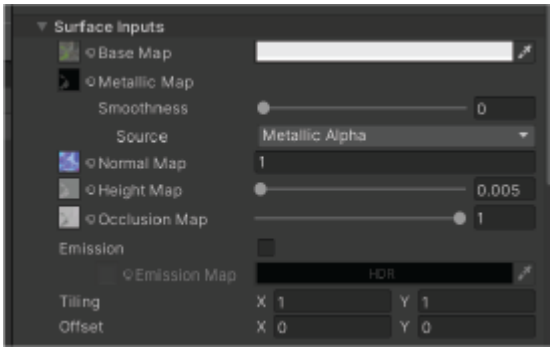
URP (Universal Render Pipeline)

Packed Map Assigned To:

`_MetallicGlossMap` and `_OcclusionMap`

Channel Layout:

- **R**: Metallic
- **A**: Smoothness
- **G**: AO (used separately)
- **B**: Not used



Key Behavior:

URP expects metallic and smoothness packed together.
AO and emission are often used as separate maps — so even if packed, they're reassigned independently.

✅ Turn Packed Mask Toggle ON:

When using URP Pipeline, **UVUnify Atlas** can generate a Packed Map Atlas
when the correct maps are present in their original materials.

: Packed Masks for URP and HDRP Pipelines

🔶 HDRP Packed Masks

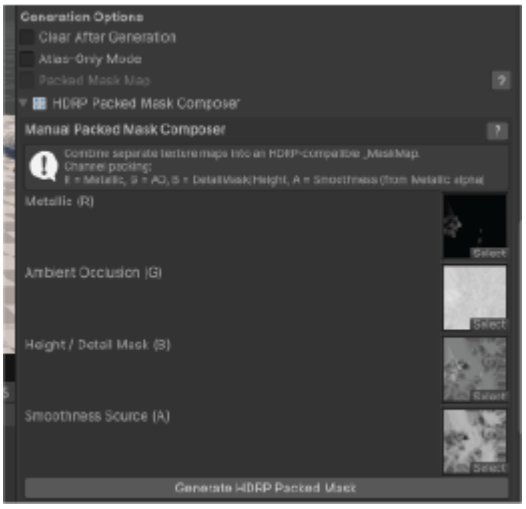
HDRP (High Definition Render Pipeline)

Packed Map Assigned To:

`_MaskMap`

Channel Layout (Strict HDRP Spec):

- **R**: Metallic
- **G**: Ambient Occlusion
- **B**: Height or Detail Mask
- **A**: Smoothness



Key Behavior:

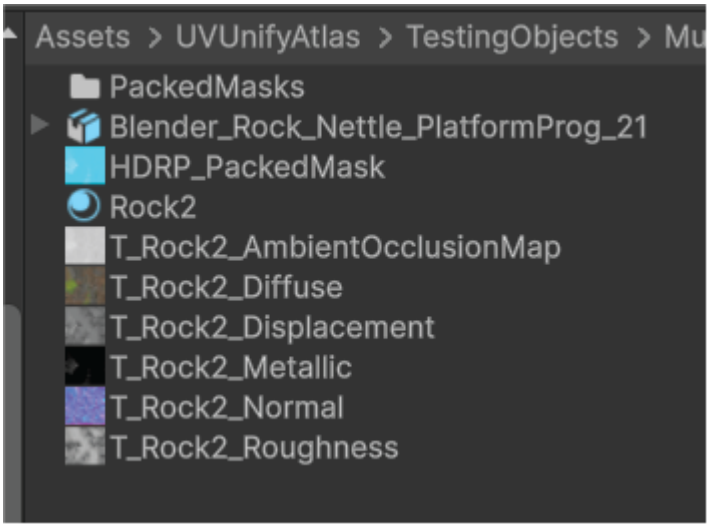
HDRP requires this exact packing and uses a single _MaskMap.
UVUnify ensures the correct alpha channel is taken from your metallic source —or lets you build one using the **HDRP Packed Mask Composer**.

🧩 Use the HDRP Packed Mask Composer:

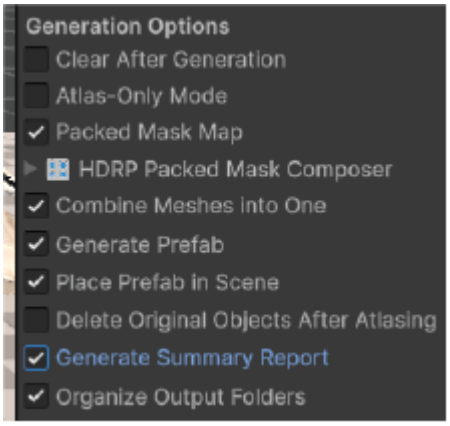
The packed Mask Composer allows you to combine maps to create a packed mask for HDRP.
UVUnify Atlas will combine atlases of all objects with materials that have packed masks.
The Packed Mask Composer puts the mask in your selected **Save Folder**.

: Packed Masks for URP and HDRP Pipelines

Packed Mask Output Workflow

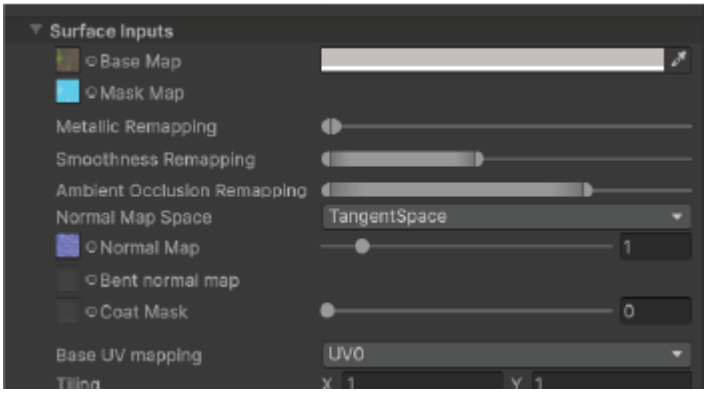


Packed Masks appear in the chosen save folder.



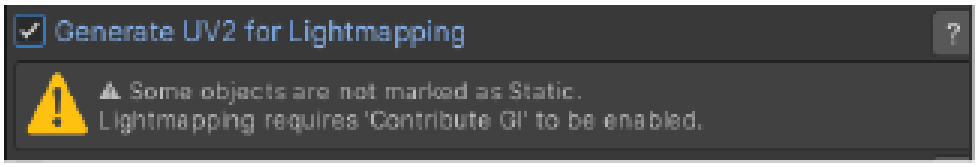
Toggle ***Packed Mask Map*** ON to generate atlases with objects using packed masks.

Packed masks created by the **Packed Mask Composer**
can be applied to the object's original material before generating **Packed Mask Atlases**.



Packed Mask Atlases generate alongside other atlases
*and can be used in the **shared combined material**.*
: UV2 for Lightmapping

Generate UV2 for Lightmapping



What It Does:

Creates UV2 (TexCoord1):
A non-overlapping UV layout specifically used by Unity's lightmapper.

Marks Mesh as Static (optional):
If the object isn't already marked Static, UVUnify can auto-mark it so Unity knows it should receive baked lighting.

Compatible with URP Only:
This feature is only useful in URP, where baked GI and shadows rely on a valid UV2 channel.

How to Use It:

Enable the toggle:
Turn on **Generate UV2 for Lightmapping** under *Generation Options*.

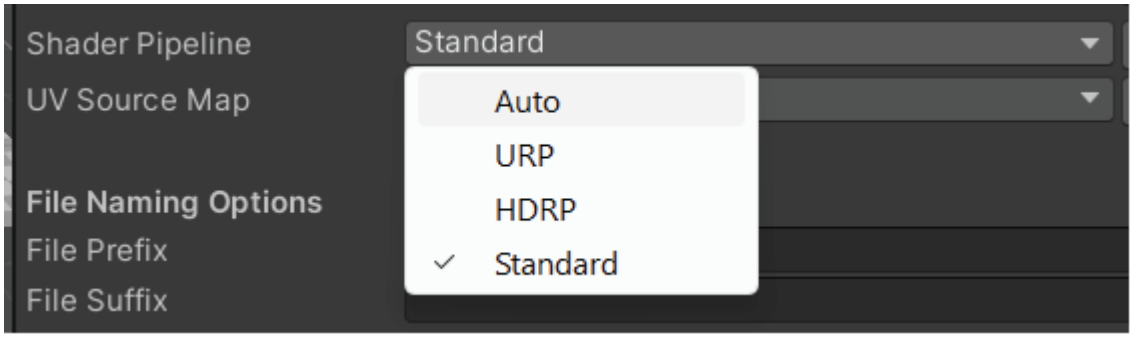
Make sure these conditions are met:

- URP pipeline is active
- Combine Meshes is also enabled
- All objects are opaque (no transparency)
- Objects use valid MeshFilters

Generate your atlases:

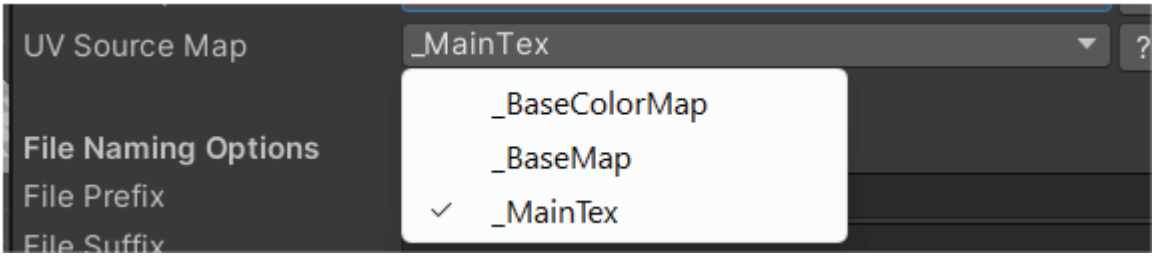
The final combined mesh will now have a properly unwrapped UV2 channel and be ready for baked lighting.
: Pipeline & UV Source Map Detection

Shader Pipeline



If no pipeline is selected, the tool will try to auto-detect it.
This may cause a blank result on first generation. Simply re-generate after the pipeline is detected to fix the issue.

UV Source Map

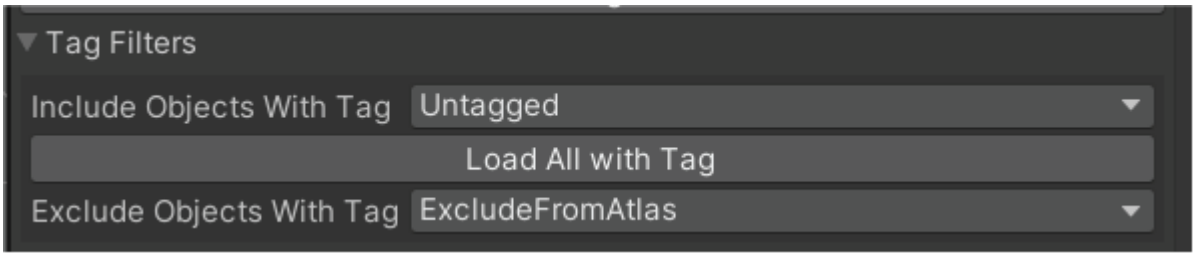


Just like Shader Pipeline, the **UV Source Map** is automatically detected by the tool based on your materials. The dropdown exists in case you want to override the detection manually.

> ***Note:***
> *If no active render pipeline is set in your project (e.g., when using Built-in/Standard), the tool defaults to* ***Standard pipeline*** *and assumes* `_MainTex` *as the UV source.*
> *: Tag Filters*

Tag Filters – Use Cases

The Tag Filters section helps you selectively load and process objects based on Unity Tags, giving you more control over which objects get atlased and which ones are ignored.



Include Objects With Tag

What it does:
Only objects with this tag will be included when you click **"Load All with Tag"**.
Default: `Untagged` — includes all objects without any specific tag.

Use Cases:

- Isolate only your environment models (e.g., tag them `Env`)
- Batch process only interactable props (e.g., tag them `Prop`)
- Avoid including lighting dummies, placeholder meshes, etc.

Exclude Objects With Tag

What it does:
Skips any object that matches this tag, even if it would otherwise be included.

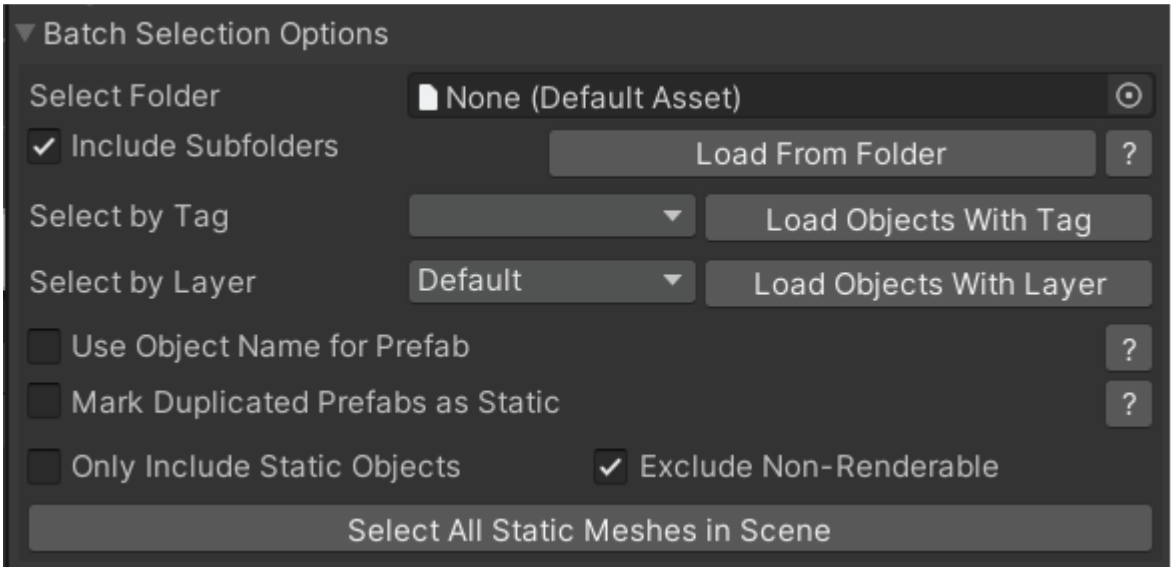
Default: `ExcludeFromAtlas` — a built-in safeguard tag your tool sets up on first use.

Use Cases:

- Avoid processing custom colliders, bounding boxes, or visual markers
- Prevent already atlased or special-case objects from being reloaded
- Useful when you're loading entire folders or scenes in bulk
  : Batch Selection

Batch Selection Options – Overview

This section lets you quickly populate the tool with objects from your project or scene based on folders, tags, layers, or scene-wide criteria — saving time when working with large scenes or asset groups.



Select Folder

Lets you choose a folder in the Project view to batch-load objects.
Use case: Load multiple prefabs from a specific folder.
When to use: Preparing atlases for entire object categories like Furniture, Props, or Environment.

Include Subfolders

If checked, loads objects recursively from all nested folders within the selected one.
Use case: Useful for well-organized asset directories that group models into subfolders by category or size.

Select by Tag / Load Objects With Tag

Lets you filter and load all scene objects with a specific tag.
Use case: Load only objects tagged Plant, Prop, Rock, etc.
Works well with the *Tag Filters* section.

Select by Layer / Load Objects With Layer

Lets you filter and load scene objects based on their layer.
Use case: Only include models from layer StaticGeometry or Interactive.

Use Object Name for Prefab

If enabled, the prefab generated during atlas processing will be named after the original object instead of a generic name.
Use case: Maintains naming clarity in larger projects.
Example: `Bench_01.prefab` instead of `CombinedMeshObject.prefab`.

Mark Duplicated Prefabs as Static

Applies Unity's Static flag to instantiated prefab copies.
Use case: Ensures the duplicated objects are compatible with static batching or lightmap baking, especially when *Only Include Static Objects* is enabled.

Only Include Static Objects

Skips any GameObjects not marked as Static in the Unity Inspector.
Use case: Useful for generating atlases for level geometry while ignoring dynamic/animated props.

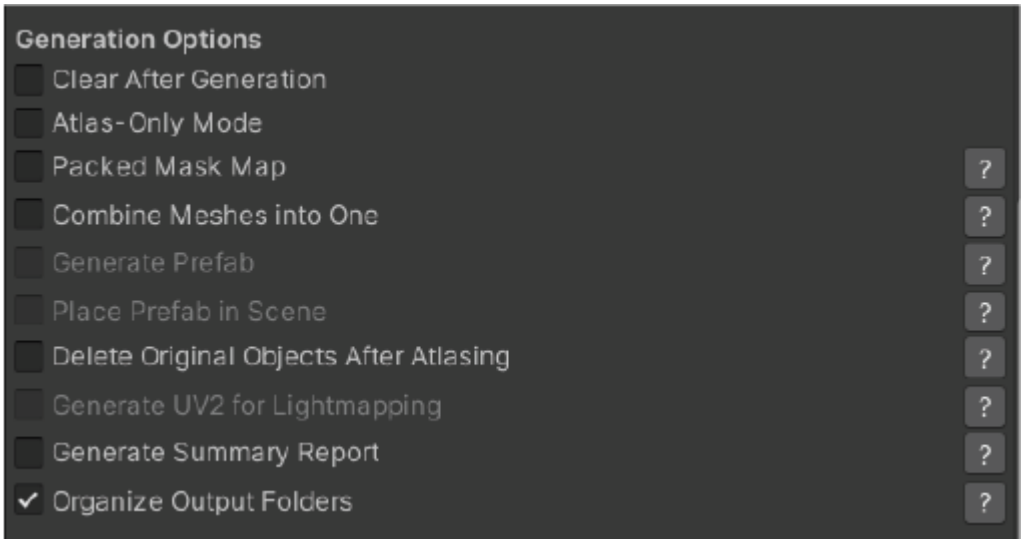
Exclude Non-Renderable

Skips any objects that don't have a MeshFilter + MeshRenderer (like empty GameObjects or triggers).
Prevents errors during processing.
Recommended ON by default.

Select All Static Meshes in Scene

Scans the entire scene and adds all GameObjects with a MeshRenderer and Static flag to the tool.
Use case: Ideal for quickly atlasing all static environment geometry before baking or optimization.

● Generation Options



Generation Options – Overview

The Generation Options section provides control over how UVUnify processes selected objects and outputs atlases, meshes, prefabs, and supporting data.
These toggles allow users to tailor the behavior of the tool based on their needs—whether they are generating texture atlases only, merging meshes for performance, creating prefabs, or cleaning up the scene afterward.

Clear After Generation

Description:
Removes temporary data or selections (e.g., clears selected objects or resets UI) after atlas generation completes.
Use Case:
Helps keep the workspace clean after a batch operation.

Atlas-Only Mode

Description:
Only generates texture atlases. Skips mesh combining, prefab generation, and other downstream steps.
Use Case:
Ideal if the user only wants to optimize materials without altering mesh hierarchy or structure.

Packed Mask Map

Description:
Generates a packed RGBA mask (e.g., Metallic-R, AO-G, Detail-B, Smoothness-A for URP).
Use Case:
Reduces texture count and improves performance for URP/HDRP pipelines by combining multiple grayscale maps into one texture.

Combine Meshes into One

Description:
Merges multiple meshes into a single combined mesh using shared material(s).
Use Case:
Improves draw call efficiency and runtime performance by reducing the number of rendered objects.

Generate Prefab

Description:
Saves the combined mesh or atlas as a reusable prefab.
Note: Disabled until "Combine Meshes into One" is enabled.
Use Case:
Stores the generated object in your project folder for future use or instantiation.

Place Prefab in Scene

Description:
Instantiates the newly created prefab into the current scene.
Use Case:
Immediate placement of the result without needing to manually drag from the Project window.

Delete Original Objects After Atlasing

Description:
Removes the original GameObjects that were combined or atlased.
Use Case:
Useful for cleanup, especially in large scenes—be cautious to avoid accidental data loss.

Generate UV2 for Lightmapping

Description:
Adds a secondary UV set (UV2) optimized for lightmapping.
Note: Typically only available when mesh combining is enabled and render pipeline supports baked lighting.
Use Case:
Necessary for baked lightmaps or static GI lighting setups.

Generate Summary Report

Description:
Outputs a log or text report detailing what was generated (e.g., number of objects processed, output paths, shader used).
Use Case:
Helpful for debugging, documentation, or automated QA workflows.

Organize Output Folders

Description:
Creates subfolders (e.g., Textures/, Materials/, Prefabs/) for storing generated assets.
Use Case:
Keeps project assets clean and organized, especially in large-scale projects.

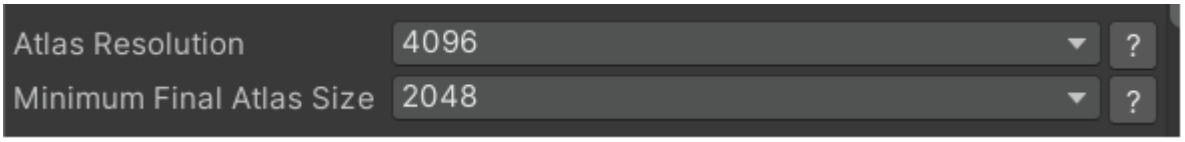
Verbose Logging

Description:
Enables detailed console logs for each step of the atlas generation process.
Use Case:
Helpful for debugging or understanding internal logic when troubleshooting issues or verifying performance.

Use Emojis in Logs

Description:
Adds emoji icons to log messages for visual clarity and user-friendly formatting.
Use Case:
Improves readability during development, especially when scanning long logs in the Console.

- **Atlas Resolution**
- **Minimum Final Atlas Size**



Atlas Resolution

Description:
Defines the maximum resolution for the generated texture atlas. Common values are 1024, 2048, 4096, or 8192.

Use Case:
Higher values allow more texture detail but may increase memory usage. Lower values may force aggressive compression or downscaling.

Minimum Final Atlas Size

Description:
Sets a lower bound for the final output atlas size after packing. Ensures that the atlas won't be too small, which could degrade visual quality.

Use Case:
Prevents Unity from automatically shrinking atlas sizes too far during optimization, ensuring a baseline level of detail.

Recommended Settings by Platform

Desktop / Console (URP or HDRP)

- **Atlas Resolution:** 4096 or 8192
- **Minimum Final Atlas Size:** 2048
- **Why:** High-res textures preserved. These platforms can handle higher memory and texture sizes, and visual fidelity is often prioritized.

Mobile (URP or Lightweight Render Pipeline)

- **Atlas Resolution:** 2048
- **Minimum Final Atlas Size:** 1024
- **Why:** Mobile GPUs have limited memory and prefer power-of-two textures ≤ 2048. Setting the minimum too high can lead to crashes on older devices.

Platform Guidelines (Continued)

VR (Standalone - Oculus/Quest/PCVR)

- **Atlas Resolution:** 2048 or 4096
- **Minimum Final Atlas Size:** 2048
- **Why:** VR needs a balance between performance and clarity. Atlases should be large enough to avoid blurring when viewed up close but not too large to hurt frame rate.

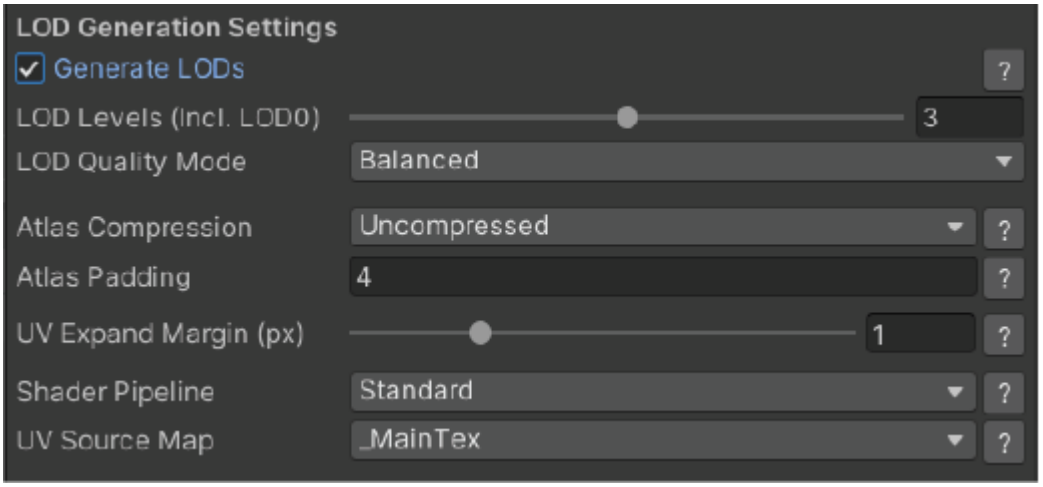
Testing / Debug Builds

- **Atlas Resolution:** 1024 or lower
- **Minimum Final Atlas Size:** 512
- **Why:** Reduces build size and texture memory for fast iteration without needing high-quality visuals.

Additional Tips

If your atlases appear blurry, try increasing **Minimum Final Atlas Size**.
If your game suffers from frame spikes or GPU memory warnings, **reduce Atlas Resolution**.

LOD Generation Settings



LOD Generation Settings – Overview

The LOD Generation Settings section controls how UVUnify generates Levels of Detail (LODs) for combined meshes.
LODs help optimize rendering by displaying simplified versions of models at different distances from the camera, reducing the rendering load without sacrificing noticeable visual quality.

Generate LODs

Description:
Enables automatic LOD generation using mesh simplification techniques.
Use case:
Improves runtime performance by reducing triangle count on distant objects. Required for projects targeting VR, mobile, or large open-world scenes.

LOD Levels (Incl. LOD0)

Description:
Sets the number of LOD levels to generate, including the original mesh (LOD0).
For example, "3" creates LOD0 (original), LOD1, and LOD2.

Typical Ranges:

- *2–3: For general use*
- *4–5: For large, highly detailed objects*

LOD Quality Mode

Description:
Controls the simplification aggressiveness and mesh fidelity:

- **High**: Minimal quality loss (slower, larger LODs)
- **Balanced**: Moderate simplification *(default)*
- **Aggressive**: Maximum reduction (best for far-distance models)

Use Case:
Choose based on how visible or performance-critical the object is.

Atlas Compression

Description:
Controls the format used for texture atlas output:

- **Uncompressed**: Highest visual fidelity, largest size
- **Compressed (Low/High)**: Reduces memory usage and improves GPU performance

Use Case:
Use **Uncompressed** for lossless texture quality or testing.
Use **Compressed** for performance and runtime efficiency.

Atlas Padding

Description:
Sets spacing (in pixels) between textures inside the atlas to prevent texture bleeding.

Typical Value:
4–8 is safe for most platforms.

UV Expand Margin (px)

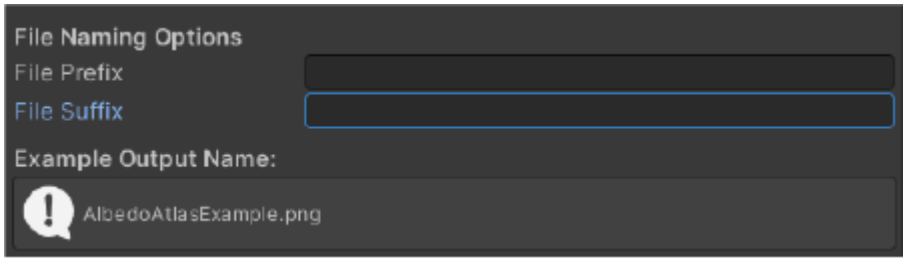
Description:
Expands the UV border edges of each packed texture before atlasing, helping prevent seams or mipmap artifacts.

Use Case:
A value of **1–2 px** is usually sufficient.
Higher values for aggressive downscaling or mipmapped LODs.

- **File Naming Options**
- **Save Folder**

File Naming Options

The File Naming Options section allows you to customize how generated asset filenames (e.g. textures, materials, prefabs) are formatted during atlas export. This ensures better organization, prevents overwriting, and helps identify asset purpose or origin at a glance.



File Prefix

Description:
A custom string added to the **start** of each generated filename.

Use Case: Helpful for categorization, such as:

- `Tree_ ---> Tree_AlbedoAtlas.png`
- `Level01_ ---> Level01_CombinedMesh.prefab`

File Suffix

Description:
A custom string added to the **end** of each filename, before the file extension.

Use Case:

- `_URP ---> AlbedoAtlas_URP.png`
- `_v2 ---> MeshCombined_v2.prefab`

Save Folder

The Save Folder section defines where all generated assets — including texture atlases, combined meshes, prefabs, and materials — will be stored in your Unity project.

- ## Features

  ### • Actions

  ### • Atlas Preview

Actions & Atlas Preview

This section is the *final execution zone* of your UVUnify tool, giving users a one-click way to run the entire pipeline and immediately preview the results.

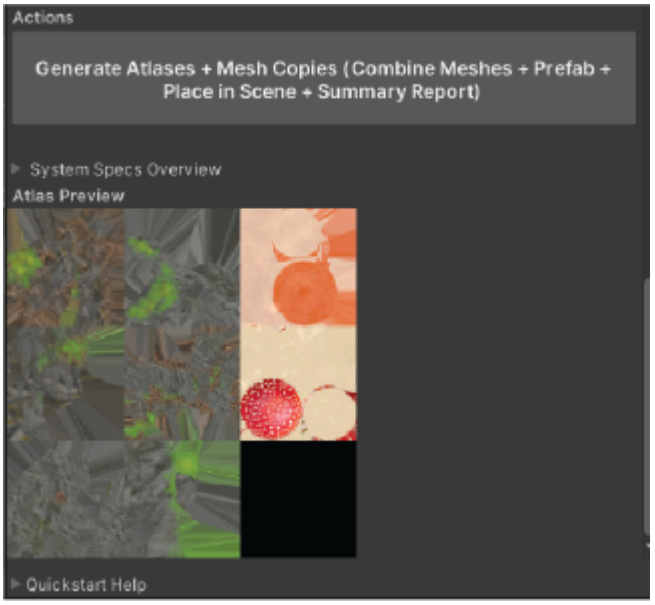
Atlas Preview

Description:
Displays a visual preview of the generated texture atlases (e.g., Albedo, Normal, MaskMap).

Use Case:
Lets users verify at a glance:

- Whether atlases packed correctly
- If anything is missing or misaligned
- Transparency issues or color mismatches

Screenshots:



● Summary Report Example

What is the Summary Report?

When enabled, **UVUnify** generates a `.txt` report listing all relevant information about the current atlas generation process. This includes:

- Shader pipeline and material transparency settings
- Mesh and atlas output details (vertex count, atlas size, padding)
- Maps detected (Albedo, Normal, AO, etc.)
- Features enabled (e.g., LODs, Prefab)

Why it matters:

This report helps document your optimization steps, makes debugging easier, and provides clear evidence of what was included or excluded — especially helpful for large projects or sharing with teammates.

Example Summary Report Output

```
==============================
UVUnify Texture Atlas Report
==============================
Generated On: 5/21/2025 8:59:11 PM

Tool Version: v1.0
Output Folder: Assets/UVUnifyAtlas/_Output
Material Asset: CombinedAtlas_Material_Opaque.mat
Group Type: Opaque
Group Suffix: _Opaque
Processed Objects: 10
Transparent Objects Detected: 0

== Material Shader & Transparency ==
Shader Pipeline: Standard
Shader Used: Standard
Material Transparency: Opaque
Auto-Set Transparency: Yes
Alpha Clipping: Disabled
Render Queue: 2000 (Opaque)
Surface Type (_Surface): 0

== Meshes & Atlas Output ==
Combined Mesh Vertex Count: 14,797
```

```
Estimated Draw Call Reduction: 10 → 1
Atlas Resolution: 4096px
Atlas Padding: 4px
UV Expand Margin: 1px
UV Source Map: _MainTex

== Maps Detected ==
Albedo: True
Normal: True
Metallic: True
AO: True
```

● FAQ & Troubleshooting

This section covers common questions and issues users may encounter when using the **UVUnify Atlas Unity Editor Tool**.

General

What does this tool do?
UVUnify Atlas helps you:

- Generate texture atlases from selected GameObjects
- Combine meshes into a single mesh
- Automatically assign a shared material based on pipeline (URP, HDRP, Standard)
- Optionally generate LODs, prefabs, UV2s, and Packed Mask Maps

---

Common Questions

Why are my output textures blank or missing?

- You may have selected the wrong shader pipeline.
- The tool will attempt to auto-detect the correct pipeline after the first run. Try running again.
- Ensure the selected objects have albedo or fallback material colors.

Why is the material missing a texture?

- Some materials use only colors, not textures. UVUnify uses a fallback color texture in those cases.
- Check the "Used Fallback Color" tag beside your object in the object list.

Why is LOD generation disabled?

- LODs can only be generated when exactly one object is selected.
- You must combine objects first, then select the combined object to generate LODs.

Why are transparent and opaque objects not combined?

- UVUnify separates opaque and transparent materials to avoid rendering issues.
- You must generate atlases for them separately.

● FAQ & Troubleshooting

● Best Practices

My materials don't look correct. What should I check?

- Confirm the correct render pipeline is selected (or leave it on **Auto**).
- Check the transparency settings: is the object marked transparent? Are you using **Cutout**?
- Make sure you increased **Atlas Padding** and **UV Expand Margin** to reduce visual artifacts.

Troubleshooting

The tool says no textures were found, but I have materials.

- Click **Refresh Selected Objects** to re-evaluate the materials.
- Make sure the materials have proper `_BaseMap` / `_MainTex` assigned.
- If you're only using color properties, UVUnify should generate **fallback textures** automatically.

Atlases are not generated or are empty.

- Ensure textures are readable.
- Make sure the save folder is valid and inside the `Assets/` directory.
- Review console output if **Verbose Logging** is enabled.

Why is the combined mesh empty?

- One or more objects may have invalid or missing UVs.
- Check the mesh filter and UV presence using the summary info box.
- You may need to force re-import or manually assign UVs in a 3D tool.

Summary report says "Skipped Maps"

- If certain maps (e.g., Emission, AO) are not detected or assigned, they will be listed under **Skipped Maps**.
- This is not necessarily an error, just an informative note.

Best Practices

- **Refresh** objects before generating if you change materials.
- Set **Shader Pipeline** to **Auto** unless you know you need something else.
- Use **4 px padding** and **1 px UV expand margin** for transparent textures to prevent bleeding.
- Enable **Generate Summary Report** to keep track of what was generated.
- If you still encounter issues, enable **Verbose Logging** and report logs from the Unity Console.

Tool Version: **v1.0** Signature: **WS-UV1**

● Folder Structure

Folder Structure Overview

When you import or use **UVUnify Atlas**, the tool organizes its assets and generated content into a clean, structured folder hierarchy to help keep your project manageable and easy to navigate.

Top-Level Folders

Folder	Purpose
`UVUnify/`	Contains any generated fallback shaders (e.g., URP/OpaqueAtlas). These are automatically created if needed based on your render pipeline.
`UVUnifyAtlas/`	Main folder containing tool assets, demo scenes, documentation, and generated outputs. This is where all editable assets and outputs are centralized.

Contents of `UVUnifyAtlas/`

Folder	Description
`_Output/`	Default location for all generated atlases, meshes, materials, prefabs, and reports.
`DemoScene/`	Optional demo scene(s) showing example usage of the tool.
`Documentation/`	PDFs, readme files, or markdowns explaining features or usage.
`Editor/`	All core editor scripts (e.g., UVUnifyAtlas.cs).
`Materials/`	Default test materials or pipeline-specific examples.
`Models/`	Optional 3D models for testing or demonstration.
`PipelineSupport/`	Optional assets for supporting URP, HDRP, or Standard pipelines.
`Textures/`	Sample textures for demonstration or quick testing.

Contents of `UVUnifyAtlas/_Output/`

Folder	Description
`Atlases/`	Packed albedo, normal, AO, and other texture atlases.
`Meshes/`	Combined mesh assets or remapped UV versions.
`Materials/`	Shared materials generated during atlas processing.
`Prefabs/`	Prefab assets saved if that option is enabled.
`Reports/`	`.txt` files with pipeline info and generation summary.
`PackedMasks/`	Optional HDRP-compatible RGBA packed mask maps.
`TextureCopies/`	Temporary readable duplicates of source textures.

● Changelog

● Version Info

Version Info & Changelog

Tool Name: UVUnify Atlas
Unity Texture Atlas Generator & Mesh Combiner

Current Version

v1.0
Release Date: **May 2025**

Changelog

v1.0 – Initial Release
A fully-featured Unity Editor tool for texture atlasing, mesh combining, and prefab creation — with support for URP, HDRP, and Standard pipelines.

Major Features

Texture Atlasing

- Packs Albedo, Normal, Metallic, AO, Height, and Emission maps
- Transparent and Opaque objects are handled separately
- Optional Packed Mask Map (URP/HDRP only)

Mesh Combining

- Merges selected GameObjects into a single mesh
- Auto UV remapping based on selected UV source
- Auto switch to 32-bit index format when needed

LOD Generator

- Custom decimation algorithm with selectable LOD quality mode
- Supports original or combined meshes
- Crossfade-enabled LODGroup prefab generation

URP Lightmapping Support

- Auto-generates UV2 lightmap coordinates for URP
- Marks generated objects as “Contribute GI” if enabled

Flexible Output

- Optional prefab generation with placement in the scene
- Automatic output folder organization
- Summary report generation with full pipeline, settings, and results

● Changelog

● Version Info

Pipeline-Aware Material Assignment

- Auto-detects project pipeline (Standard, URP, HDRP)
- Uses compatible shaders or generates stub shaders if needed
- Supports custom material transparency and alpha clipping logic

---

Editor UI Enhancements

- Batch selection by tag, layer, or folder
- Toggleable emojis, verbose logging, system specs overview
- Save folder override with auto-default to `_Output`

Shader Compatibility

UVUnify Atlas supports URP (Lit), HDRP (Lit), and Built-in (Standard) shaders.
If a required shader is missing, a fallback Standard shader will be used.
If a named custom shader (like `UVUnify/URP/OpaqueAtlas`) is not found, a stub shader is auto-generated in the `UVUnify/Shaders` folder to prevent missing material errors.

Known Limitations (v1.0)

- Does not support SkinnedMeshRenderers or animated meshes
- Manual setup recommended for custom shader graphs or unlit materials
- Atlas generation assumes UV0 layout (non-overlapping)

Tested With

Unity Versions	Render Pipelines	Platforms
2020.3 LTS, Unity 6.0+	URP 12+, HDRP 12+, Built-in Render Pipeline	Windows (Editor), macOS (Editor)

> *Runtime use is not supported — this is an Editor-only tool.*

Recommended System Requirements

- **Minimum:** 4-core CPU, 8 GB RAM
- **Recommended:** 6-core CPU, 16+ GB RAM, 3+ GB VRAM
- Atlas resolution above 8192 may require high-end GPU